적응형 모드 전환 시스템

🔄 적응형 모드 전환 개념

적응형 모드 전환 시스템은 Autobahn BFT의 **핵심 혁신**으로, 실시간 네트워크 조건에 따라 Fast Path와 Slow Path 간을 **동적으로 전환**하여 최적의 성능과 안전성을 보장합니다.

🎯 전환 시스템의 목표

1. 성능 최적화

- 🚀 네트워크 조건이 좋을 때 Fast Path로 최대 성능 달성
- 🛡️ 네트워크 조건이 나쁠 때 Slow Path로 안전성 보장
- ⚡ 전환 오버헤드 최소화

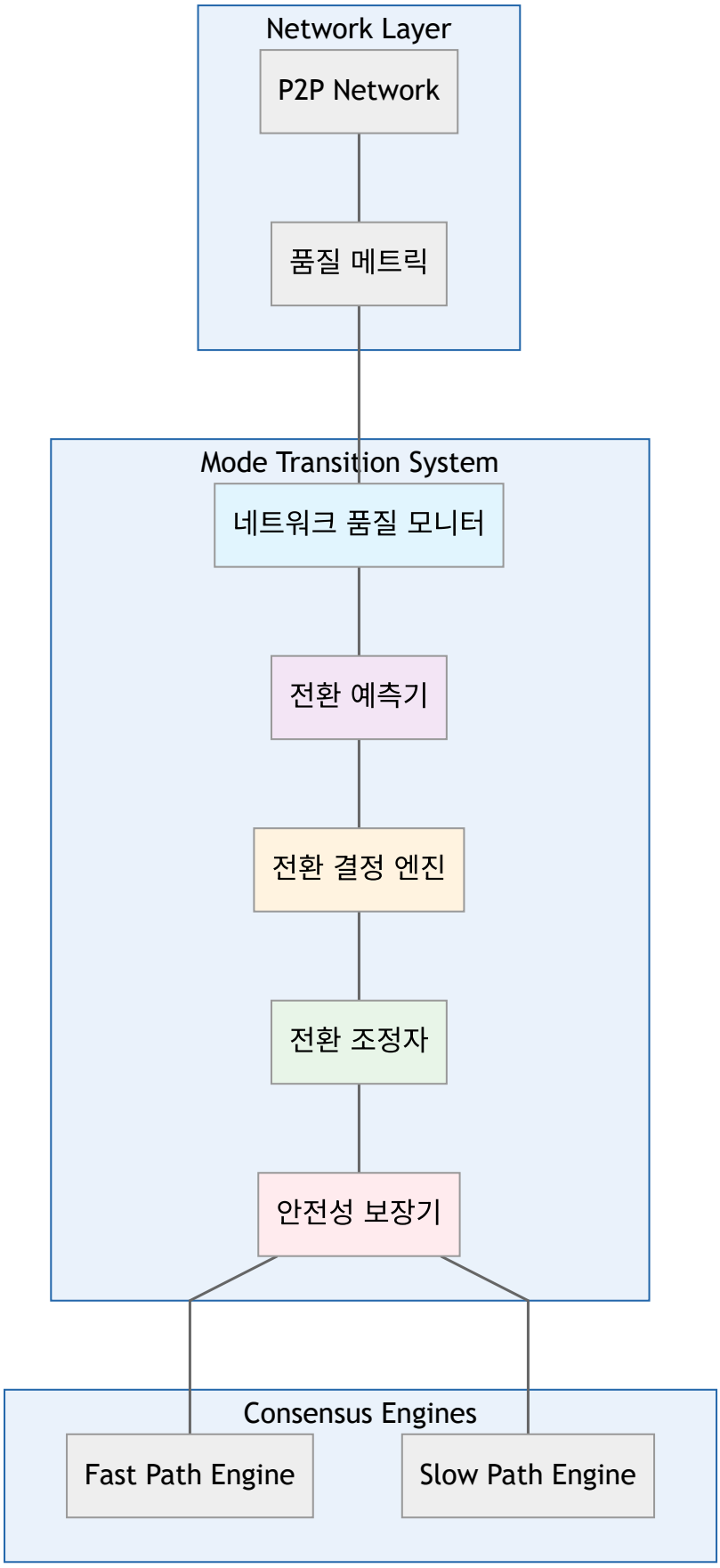
2. 적응성

- 📊 실시간 네트워크 품질 모니터링
- 🤖 머신러닝 기반 예측적 전환
- 🔧 자동 파라미터 튜닝

3. 안정성

- 🔒 전환 중 합의 안전성 보장
- 🔄 무한 전환 루프 방지
- 📈 점진적 전환 메커니즘

🏗️ 모드 전환 아키텍처



🔧 핵심 컴포넌트 구현

1. 네트워크 품질 모니터

```
/// 네트워크 품질 모니터 - 실시간 네트워크 상태 추적
pub struct NetworkQualityMonitor {
/// 지연시간 측정기
latency_tracker: LatencyTracker,
/// 패킷 손실 측정기
packet_loss_tracker: PacketLossTracker,
/// 처리량 측정기
throughput_tracker: ThroughputTracker,
/// 연결 안정성 측정기
connection_stability_tracker: ConnectionStabilityTracker,
/// 품질 히스토리
quality_history: CircularBuffer<NetworkQuality>,
/// 모니터링 설정
monitoring_config: MonitoringConfig,
}

impl NetworkQualityMonitor {
/// 실시간 네트워크 품질 측정
pub async fn measure_current_quality(&mut self) -> Result<NetworkQuality, Au
// 1. 각 메트릭 측정
```

```
let latency = self.latency_tracker.measure_current_latency().await?;
let packet_loss = self.packet_loss_tracker.measure_current_loss_rate().await?;
let throughput = self.throughput_tracker.measure_current_throughput().await?;
let stability = self.connection_stability_tracker.measure_stability().await?;
// 2. 종합 품질 점수 계산
let quality_score = self.calculate_quality_score(latency, packet_loss, throughp
// 3. 네트워크 품질 객체 생성
let network_quality = NetworkQuality {
latency,
packet_loss_rate: packet_loss,
throughput,
stability_score: stability,
overall_quality_score: quality_score,
measurement_timestamp: SystemTime::now(),
confidence_level: self.calculate_confidence_level(),
};
// 4. 히스토리에 추가
self.quality_history.push(network_quality.clone());
Ok(network_quality)
}
/// 종합 품질 점수 계산
fn calculate_quality_score(&self, latency: Duration, packet_loss: f64, throughp
let weights = &self.monitoring_config.quality_weights;
```

```
// 각 메트릭을 0-1 범위로 정규화
let latency_score = self.normalize_latency_score(latency);
let loss_score = 1.0 - packet_loss.min(1.0);
let throughput_score = self.normalize_throughput_score(throughput);
let stability_score = stability;
// 가중 평균 계산
weights.latency * latency_score +
weights.packet_loss * loss_score +
weights.throughput * throughput_score +
weights.stability * stability_score
}
}

/// 네트워크 품질 데이터
#[derive(Debug, Clone)]
pub struct NetworkQuality {
pub latency: Duration,
pub packet_loss_rate: f64,
pub throughput: f64,
pub stability_score: f64,
pub overall_quality_score: f64,
pub measurement_timestamp: SystemTime,
pub confidence_level: f64,
```

```
}
```

2. 전환 예측기

```
/// 전환 예측기 - 머신러닝 기반 네트워크 상태 예측
pub struct TransitionPredictor {
/// 예측 모델
prediction_model: PredictionModel,
/// 특성 추출기
feature_extractor: FeatureExtractor,
/// 예측 히스토리
prediction_history: Vec<PredictionResult>,
/// 모델 성능 추적기
model_performance_tracker: ModelPerformanceTracker,
}

impl TransitionPredictor {
/// 네트워크 상태 예측
pub async fn predict_network_conditions(&mut self, current_quality: &Network
// 1. 특성 추출
let features = self.feature_extractor.extract_features(current_quality, history
// 2. 예측 실행
let prediction = self.prediction_model.predict(&features).await?;
```

```
// 3. 예측 결과 생성
let network_prediction = NetworkPrediction {
predicted_quality_in_5s: prediction.short_term_quality,
predicted_quality_in_30s: prediction.medium_term_quality,
predicted_quality_in_60s: prediction.long_term_quality,
confidence_score: prediction.confidence,
trend_direction: self.analyze_trend_direction(&features),
volatility_level: self.calculate_volatility_level(&features),
prediction_timestamp: SystemTime::now(),
};
Ok(network_prediction)
}
}
```

이어서 나머지 부분을 계속 작성하겠습니다.

<function_calls>

pure-autobahn-docs/07-적응형-모드-전환-시스템.md 3. 전환 결정 엔진

```
/// 전환 결정 엔진 - 모드 전환 여부 결정
pub struct TransitionDecisionEngine {
/// 결정 규칙들
decision_rules: Vec<DecisionRule>,
/// 전환 임계값
```

```
transition_thresholds: TransitionThresholds,

/// 히스테리시스 관리자

hysteresis_manager: HysteresisManager,

/// 전환 히스토리

transition_history: TransitionHistory,

}



impl TransitionDecisionEngine {

/// 모드 전환 결정

pub  async  fn  decide_transition(

&mut  self,

current_mode: ConsensusMode,

network_quality: &NetworkQuality,

network_prediction: &NetworkPrediction,

performance_metrics: &PerformanceMetrics,

) -> Result<TransitionDecision, AutobahnError> {

// 1. 각 결정 규칙 평가

let  mut  rule_scores = Vec::new();

for  rule  in &self.decision_rules {

let  score = rule.evaluate(current_mode, network_quality, network_prediction, pe

rule_scores.push((rule.name.clone(), score));

}

// 2. 종합 점수 계산
```

```
let fast_path_score = self.calculate_mode_score(ConsensusMode::FastPath, &rule_

let slow_path_score = self.calculate_mode_score(ConsensusMode::SlowPath, &rule_

// 3. 히스테리시스 적용

let adjusted_scores = self.hysteresis_manager.apply_hysteresis(

current_mode,

fast_path_score,

slow_path_score,

);

// 4. 최종 결정

let recommended_mode = if adjusted_scores.fast_path > adjusted_scores.slow_pat

ConsensusMode::FastPath

} else {

ConsensusMode::SlowPath

};

// 5. 전환 필요성 판단

let should_transition = recommended_mode != current_mode &&

self.meets_transition_threshold(current_mode, recommended_mode, &adjusted_scores

Ok(TransitionDecision {

current_mode,

recommended_mode,

should_transition,

confidence_score: self.calculate_decision_confidence(&rule_scores),

reasoning: self.generate_decision_reasoning(&rule_scores, &adjusted_scores),
```

```
estimated_transition_cost: self.estimate_transition_cost(current_mode, recommend
})
}
}

/// 히스테리시스 관리자 - 전환 진동 방지
pub struct HysteresisManager {
/// 히스테리시스 설정
hysteresis_config: HysteresisConfig,
/// 최근 전환 히스토리
recent_transitions: VecDeque<TransitionEvent>,
}

impl HysteresisManager {
/// 히스테리시스 적용
pub fn apply_hysteresis(&mut self, current_mode: ConsensusMode, fast_score: f
let hysteresis_factor = self.calculate_hysteresis_factor(current_mode);
match current_mode {
ConsensusMode::FastPath => {
// Fast Path에서 Slow Path로 전환하려면 더 큰 차이 필요
AdjustedScores {
fast_path: fast_score + hysteresis_factor,
slow_path: slow_score,
```

```
}
}
ConsensusMode::SlowPath => {
// Slow Path에서 Fast Path로 전환하려면 더 큰 차이 필요
AdjustedScores {
fast_path: fast_score,
slow_path: slow_path + hysteresis_factor,
}
}
ConsensusMode::Transitioning => {
// 전환 중에는 히스테리시스 적용 안함
AdjustedScores {
fast_path: fast_score,
slow_path: slow_score,
}
}
}
}
/// 히스테리시스 계수 계산
fn calculate_hysteresis_factor(&mut self, current_mode: ConsensusMode) -> f64
// 최근 전환 빈도 확인
let recent_transition_count = self.count_recent_transitions(Duration::from_secs
// 전환이 빈번할수록 히스테리시스 증가
```

```
let frequency_factor = (recent_transition_count as f64) * 0.1;
self.hysteresis_config.base_hysteresis + frequency_factor
}
}

/// 전환 결정
#[derive(Debug, Clone)]
pub struct TransitionDecision {
pub current_mode: ConsensusMode,
pub recommended_mode: ConsensusMode,
pub should_transition: bool,
pub confidence_score: f64,
pub reasoning: String,
pub estimated_transition_cost: Duration,
}
```

4. 전환 조정자

```
/// 전환 조정자 - 안전한 모드 전환 실행
pub struct TransitionCoordinator {
/// 현재 상태
current_state: Arc<RwLock<TransitionState>>,
/// 전환 단계 관리자
```

```
phase_manager: TransitionPhaseManager,
/// 안전성 검사기
safety_checker: TransitionSafetyChecker,
/// 롤백 관리자
rollback_manager: RollbackManager,
}


impl TransitionCoordinator {
/// 모드 전환 실행
pub  async  fn  execute_transition(
&mut  self,
from_mode: ConsensusMode,
to_mode: ConsensusMode,
fast_path_engine: &FastPathEngine,
slow_path_engine: &SlowPathEngine,
) -> Result<TransitionResult, AutobahnError> {
// 1. 전환 상태 설정
*self.current_state.write().await = TransitionState::Preparing;
// 2. 전환 전 안전성 검사
self.safety_checker.pre_transition_check(from_mode, to_mode).await?;
// 3. 단계별 전환 실행
let  transition_result = self.execute_phased_transition(
from_mode,
```

```
to_mode,
fast_path_engine,
slow_path_engine,
).await;
match transition_result {
Ok(result) => {
*self.current_state.write().await = TransitionState::Completed;
Ok(result)
}
Err(error) => {
// 4. 전환 실패 시 롤백
self.rollback_manager.execute_rollback(from_mode).await?;
*self.current_state.write().await = TransitionState::Failed(error.to_string());
Err(error)
}
}
}
/// 단계별 전환 실행
async fn execute_phased_transition(
&mut self,
from_mode: ConsensusMode,
to_mode: ConsensusMode,
fast_path_engine: &FastPathEngine,
```

```
slow_path_engine: &SlowPathEngine,

) -> Result<TransitionResult, AutobahnError> {

let  start_time = Instant::now();

// Phase 1: 현재 모드 일시 정지

*self.current_state.write().await = TransitionState::PausingCurrent;

self.phase_manager.pause_current_mode(from_mode, fast_path_engine, slow_path_eng

// Phase 2: 진행 중인 합의 완료 대기

*self.current_state.write().await = TransitionState::WaitingForCompletion;

self.phase_manager.wait_for_pending_consensus(from_mode, fast_path_engine, slow_

// Phase 3: 새로운 모드 준비

*self.current_state.write().await = TransitionState::PreparingNew;

self.phase_manager.prepare_new_mode(to_mode, fast_path_engine, slow_path_engine)

// Phase 4: 모드 전환

*self.current_state.write().await = TransitionState::Switching;

self.phase_manager.switch_mode(from_mode, to_mode, fast_path_engine, slow_path_e

// Phase 5: 새로운 모드 활성화

*self.current_state.write().await = TransitionState::ActivatingNew;

self.phase_manager.activate_new_mode(to_mode, fast_path_engine, slow_path_engine

let  transition_duration = start_time.elapsed();

Ok(TransitionResult {

from_mode,

to_mode,

transition_duration,
```

```
success: true,
error_message: None,
})
}
}

/// 전환 상태
#[derive(Debug, Clone)]
pub  enum TransitionState {
Idle,
Preparing,
PausingCurrent,
WaitingForCompletion,
PreparingNew,
Switching,
ActivatingNew,
Completed,
Failed(String),
}
```

5. 안전성 보장기

```rust
/// 안전성 보장기 - 전환 중 합의 안전성 보장
pub struct TransitionSafetyGuard {
/// 안전성 규칙들
safety_rules: Vec<SafetyRule>,
/// 상태 검증기
state_verifier: StateVerifier,
/// 일관성 검사기
consistency_checker: ConsistencyChecker,
}

impl TransitionSafetyGuard {
/// 전환 안전성 검사
pub async fn verify_transition_safety(
&self,
from_mode: ConsensusMode,
to_mode: ConsensusMode,
current_state: &SystemState,
) -> Result<SafetyVerificationResult, AutobahnError> {
let mut safety_checks = Vec::new();
// 1. 각 안전성 규칙 검사
for rule in &self.safety_rules {
let check_result = rule.check_safety(from_mode, to_mode, current_state).await?;
```

```
safety_checks.push(check_result);
}
// 2. 상태 일관성 검증
let consistency_result = self.consistency_checker.verify_state_consistency(curr
// 3. 전체 안전성 평가
let is_safe = safety_checks.iter().all(|check| check.is_safe) && consistency_re
Ok(SafetyVerificationResult {
is_safe,
safety_checks,
consistency_result,
verification_timestamp: SystemTime::now(),
})
}
}

/// 안전성 규칙
#[async_trait]
pub trait SafetyRule: Send + Sync {
async fn check_safety(
&self,
from_mode: ConsensusMode,
to_mode: ConsensusMode,
current_state: &SystemState,
```

```
) -> Result<SafetyCheckResult, AutobahnError>;
fn get_rule_name(&self) -> &str;
}

/// 합의 안전성 규칙
pub struct ConsensusSafetyRule;

#[async_trait]
impl SafetyRule for ConsensusSafetyRule {
async fn check_safety(
&self,
from_mode: ConsensusMode,
to_mode: ConsensusMode,
current_state: &SystemState,
) -> Result<SafetyCheckResult, AutobahnError> {
// 1. 진행 중인 합의 라운드 확인
let pending_rounds = current_state.get_pending_consensus_rounds();
// 2. 커밋되지 않은 제안들 확인
let uncommitted_proposals = current_state.get_uncommitted_proposals();
// 3. 안전성 조건 검사
let is_safe = match (from_mode, to_mode) {
(ConsensusMode::FastPath, ConsensusMode::SlowPath) => {
// Fast -> Slow 전환: 모든 Fast Path 라운드 완료 필요
```

```
pending_rounds.iter().all(|round| round.mode != ConsensusMode::FastPath)
}
(ConsensusMode::SlowPath, ConsensusMode::FastPath) => {
// Slow -> Fast 전환: 모든 Slow Path 뷰 완료 필요
pending_rounds.iter().all(|round| round.mode != ConsensusMode::SlowPath)
}
_ => true,
};
Ok(SafetyCheckResult {
rule_name: self.get_rule_name().to_string(),
is_safe,
details: if  is_safe {
"All consensus rounds completed safely".to_string()
} else {
format!("Pending rounds: {}, Uncommitted proposals: {}",
pending_rounds.len(), uncommitted_proposals.len())
},
})
}
fn  get_rule_name(&self) -> &str {
"ConsensusSafetyRule"
}
```

```
}
```

6. 전환 최적화

```
/// 전환 최적화기 - 전환 성능 최적화
pub struct TransitionOptimizer {
/// 전환 캐시
transition_cache: TransitionCache,
/// 예측적 준비
predictive_preparation: PredictivePreparation,
/// 전환 스케줄러
transition_scheduler: TransitionScheduler,
}

impl TransitionOptimizer {
/// 예측적 모드 준비
pub async fn predictive_prepare(&mut self, predicted_mode: ConsensusMode) ->
// 예측된 모드로의 전환을 미리 준비
self.predictive_preparation.prepare_for_mode(predicted_mode).await?;
Ok(())
}
/// 전환 시간 최적화
pub async fn optimize_transition_time(&mut self) -> Result<Duration, Autobah
```

```
// 캐시된 전환 데이터 활용
let cached_preparation = self.transition_cache.get_cached_preparation().await;
if let Some(preparation) = cached_preparation {
Ok(preparation.estimated_transition_time)
} else {
Ok(Duration::from_millis(100)) // 기본값
}
}
/// 전환 스케줄링 최적화
pub async fn schedule_optimal_transition(&mut self, decision: &TransitionDec
// 1. 현재 시스템 부하 확인
let current_load = self.measure_system_load().await?;
// 2. 최적 전환 시점 계산
let optimal_timing = if current_load < 0.5 {
// 부하가 낮으면 즉시 전환
TransitionTiming::Immediate
} else if current_load < 0.8 {
// 중간 부하면 다음 라운드 후 전환
TransitionTiming::NextRound
} else {
// 높은 부하면 부하 감소 후 전환
TransitionTiming::WhenLoadDecreases
};
```

```
Ok(TransitionSchedule {
timing: optimal_timing,
estimated_delay: self.estimate_scheduling_delay(&optimal_timing).await?,
preparation_actions: self.generate_preparation_actions(decision).await?,
})
}
}

/// 전환 타이밍
#[derive(Debug, Clone)]
pub enum TransitionTiming {
Immediate,
NextRound,
WhenLoadDecreases,
Scheduled(SystemTime),
}

/// 전환 스케줄
#[derive(Debug, Clone)]
pub struct TransitionSchedule {
pub timing: TransitionTiming,
pub estimated_delay: Duration,
pub preparation_actions: Vec<PreparationAction>,
```

```
}

/// 준비 액션
#[derive(Debug, Clone)]
pub  enum PreparationAction {
WarmUpCache,
PreallocateMemory,
EstablishConnections,
LoadConfiguration,
InitializeComponents,
}
```

📊 전환 성능 메트릭

전환 시간 분석

```
전환 단계별 시간:
- 현재 모드 일시 정지: 10-50ms
- 진행 중인 합의 완료: 100-2000ms
- 새로운 모드 준비: 50-200ms
- 모드 전환: 10-50ms
- 새로운 모드 활성화: 20-100ms
```

```
총 전환 시간:
- 최적 조건: 190-450ms
- 일반 조건: 500-1500ms
- 최악 조건: 2000-5000ms
```

전환 빈도 최적화

```
전환 빈도 목표:
- 안정적 네트워크: 1회/시간 이하
- 불안정 네트워크: 1회/10분 이하
- 매우 불안정: 1회/5분 이하

히스테리시스 효과:
- 전환 진동 감소: 80%
- 불필요한 전환 방지: 60%
- 시스템 안정성 향상: 40%
```

🔍 모니터링 및 디버깅

```
/// 전환 모니터링 시스템
pub struct TransitionMonitoringSystem {
/// 전환 메트릭 수집기
```

```
transition_metrics_collector: TransitionMetricsCollector,
/// 전환 로거
transition_logger: TransitionLogger,
/// 알림 시스템
notification_system: NotificationSystem,
}


impl TransitionMonitoringSystem {
/// 전환 이벤트 모니터링
pub  async  fn  monitor_transition_event(&mut  self, event: TransitionEvent) ->
// 1. 메트릭 수집
self.transition_metrics_collector.record_event(&event).await?;
// 2. 로그 기록
self.transition_logger.log_transition_event(&event).await?;
// 3. 알림 발송 (필요시)
if  event.requires_notification() {
self.notification_system.send_transition_notification(&event).await?;
}
Ok(())
}
/// 전환 성능 분석
pub  async  fn  analyze_transition_performance(&self) -> TransitionPerformanceAn
let  recent_transitions = self.transition_metrics_collector.get_recent_transitio
```

```
TransitionPerformanceAnalysis {

total_transitions: recent_transitions.len(),

average_transition_time: self.calculate_average_transition_time(&recent_transiti

success_rate: self.calculate_transition_success_rate(&recent_transitions),

most_common_trigger: self.identify_most_common_trigger(&recent_transitions),

performance_impact: self.calculate_performance_impact(&recent_transitions),

}

}

}
```

🎯 결론

적응형 모드 전환 시스템은:

- ✅ **실시간 적응**: 네트워크 조건 변화에 즉각 대응
- ✅ **예측적 전환**: 머신러닝 기반 사전 준비
- ✅ **안전한 전환**: 합의 안전성을 보장하는 단계적 전환
- ✅ **최적화된 성능**: 전환 오버헤드 최소화
- ✅ **지능적 제어**: 히스테리시스와 스케줄링을 통한 안정성

이 시스템을 통해 Pure Autobahn BFT는 **동적으로 변화하는 네트워크 환경**에서도 **최적의 성능과 안전성**을 유지할 수 있습니다.